

图 2.1.2 建立 1 位全加器 add1bit 项目

点击 Next 按钮进入 Project Type 界面，选择 Empty project 选项，点击 Next 按钮进入 Add Files 界面，不选择任何文件，再点击 Next 按钮进入器件选择(Family,Device & Board Settings)界面，由于 DE10-Lite 开发板采用的 FPGA 芯片的型号是 MAX10 系列 10M50DAF484C7G，因此器件选择界面的设置如图 2.1.3 所示：

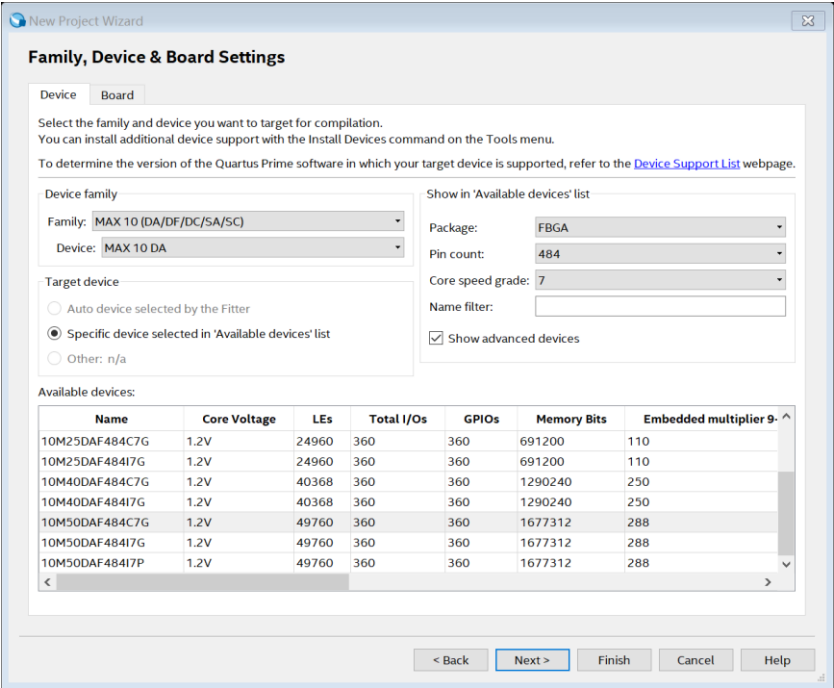


图 2.1.3 1 位全加器 add1bit 项目芯片选择

点击 Next 按钮进入 EDA Tool Settings 界面和 Summary 界面，由于在这两个界面均不需要进行设置，可点击 Next 按钮然后点击 Finish 按钮完成新项目的设置向导。

选择 File>New...菜单命令，设计文件类型选择 VHDL File。进入到项目设计界面，将缺省文件名 Vhdl.vhd 另存为(File>Save As...)文件 add1bit.vhd，输入 1 位全加器的 VHDL 源程序后如图 2.1.4 所示。

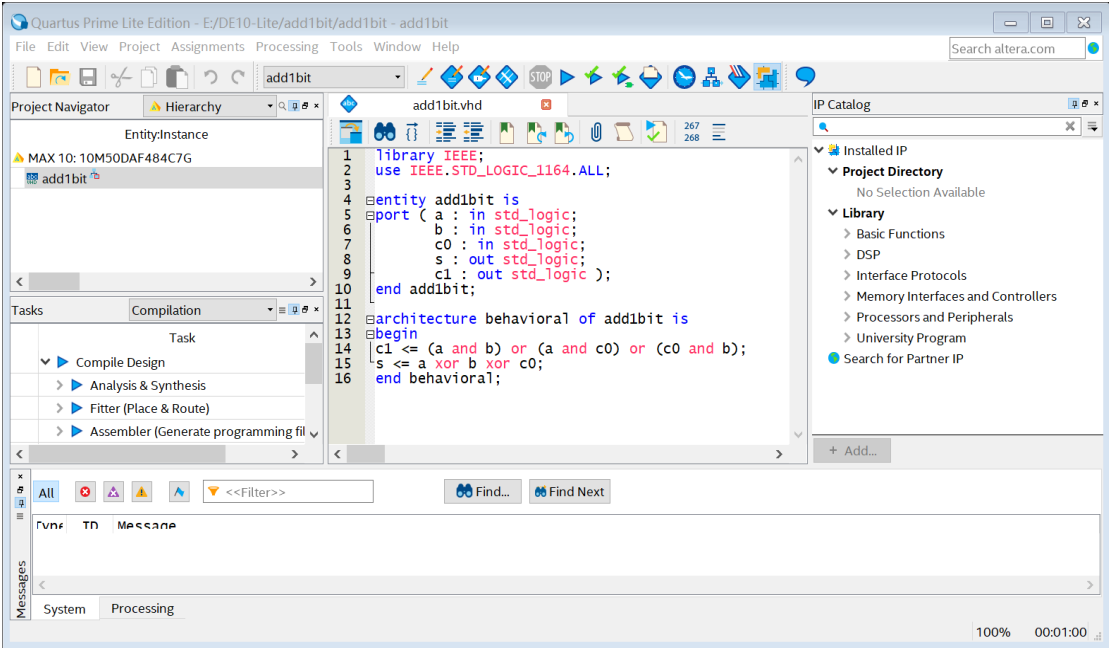


图 2.1.4 1 位全加器 add1bit 项目文件

执行 **Processing>Start Compilation** 菜单命令，程序会运行稍长时间，运行结束后界面如图 2.1.5 所示。

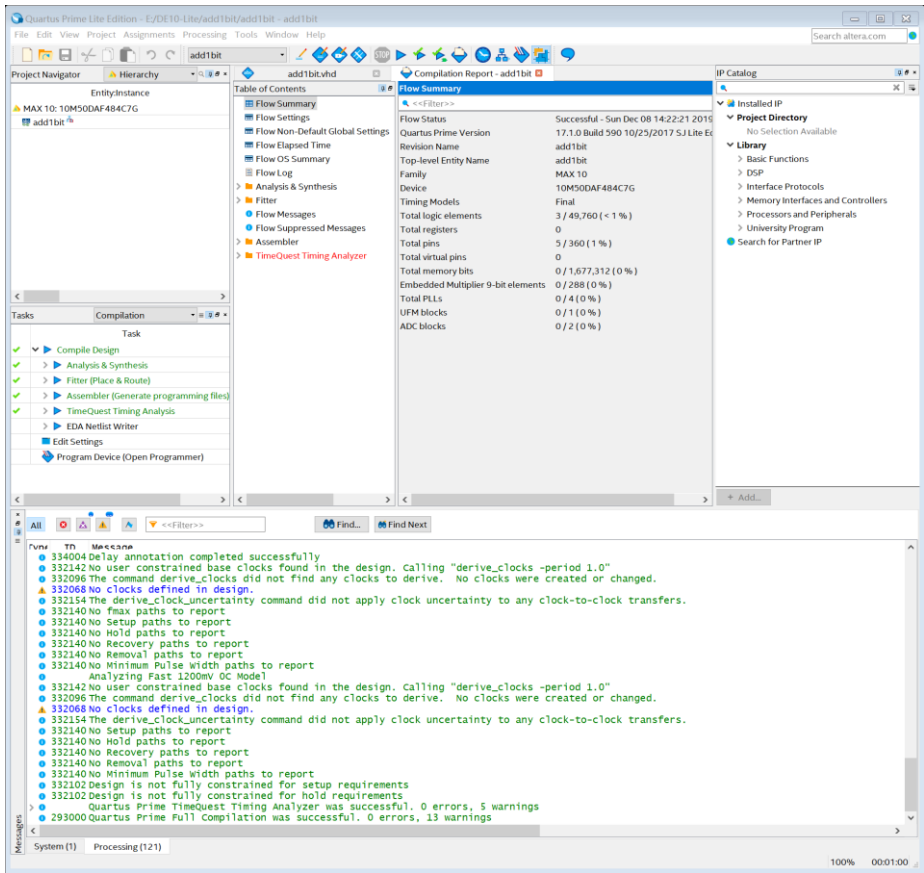


图 2.1.5 程序编译界面

如果在编译过程中发生错误，编译停止，编译的错误信息会在编译界面的下部显示，可以根据显示的信息查找错误。编译结果必须是 0 个 errors，有若干个 warnings 可以忽略。

执行 **Assignments>Pin planner** 菜单命令，进行引脚分配后的引脚分配编辑器窗口如图 2.1.6 所示。在这里将 3 个拨码开关 SW2、SW1、SW0 作为全加器的输入，将 2 个 LED 灯 LEDR1、LEDR0 作为全加器的输出。

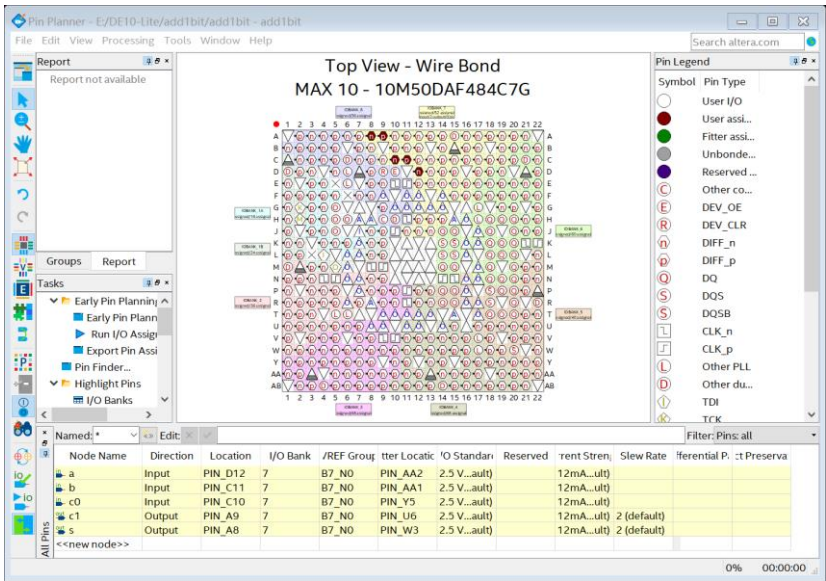


图 2.1.6 1 位全加器引脚分配编辑器窗口

紧接着进行未用引脚的设置,执行 **Assignments>Device** 菜单命令,继续点击 **Device and Pin Options...** 按键, 出现 **Device and Pin Options** 设置窗口, 选中 **Unused Pins** 条目, 在 **Reserve all unused pins** 下拉列表中选择 **As input tri-stated** 设置项, 点击 **OK** 按键返回主界面。

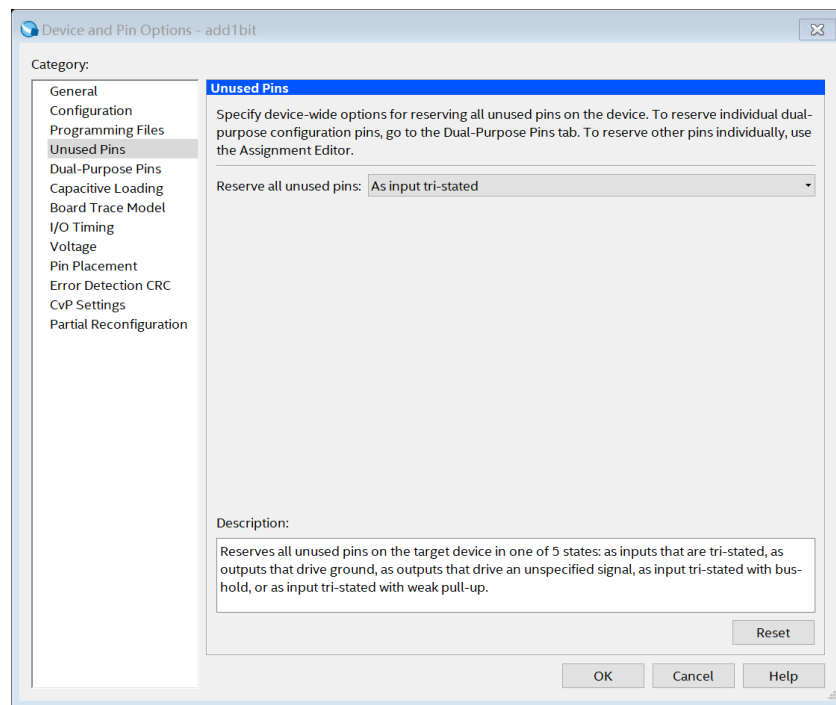


图 2.1.7 未用引脚设置

重新对项目进行完整编译后生成 **add1bit.sof** 文件,经程序下载后就可以在 **DE10-Lite** 开发板上验证程序的正确与否了。

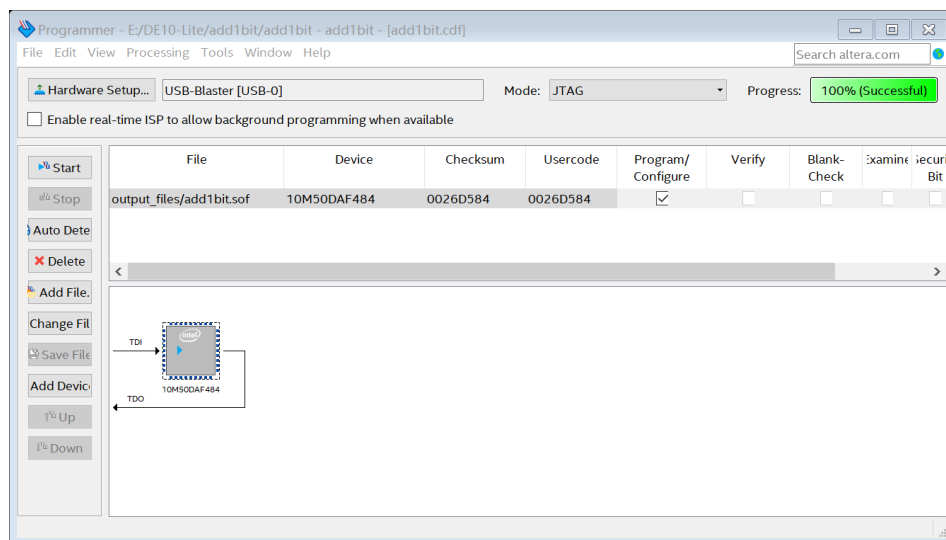


图 2.1.8 1 位全加器程序下载

2.2 DE10-Lite 开发板数字逻辑设计实验 - 4 位全加器实验

4 位全加器的实现原理如下图所示：

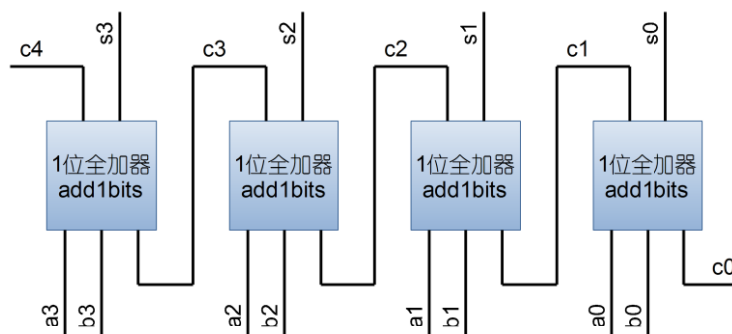


图 2.2.1 4 位全加器实现原理

从图中可以看出，4 位全加器由 4 个 1 位全加器级联而成，把 4 位全加器作为一个顶层设计，1 位全加器就是一个底层设计，相当与一个子程序。在 VHDL 文件中可以用元件说明(component)来表述。

在启动 Quartus Prime Lite Edition 集成开发软件后，按建立项目向导的 5 个步骤建立 add4bit 项目，项目文件夹为 E:\DE10-Lite\add4bit。选择 File>New...菜单命令，设计文件类型选择 VHDL File。进入到项目设计界面，输入 4 位全加器的 VHDL 源程序后如图 2.2.2 所示。

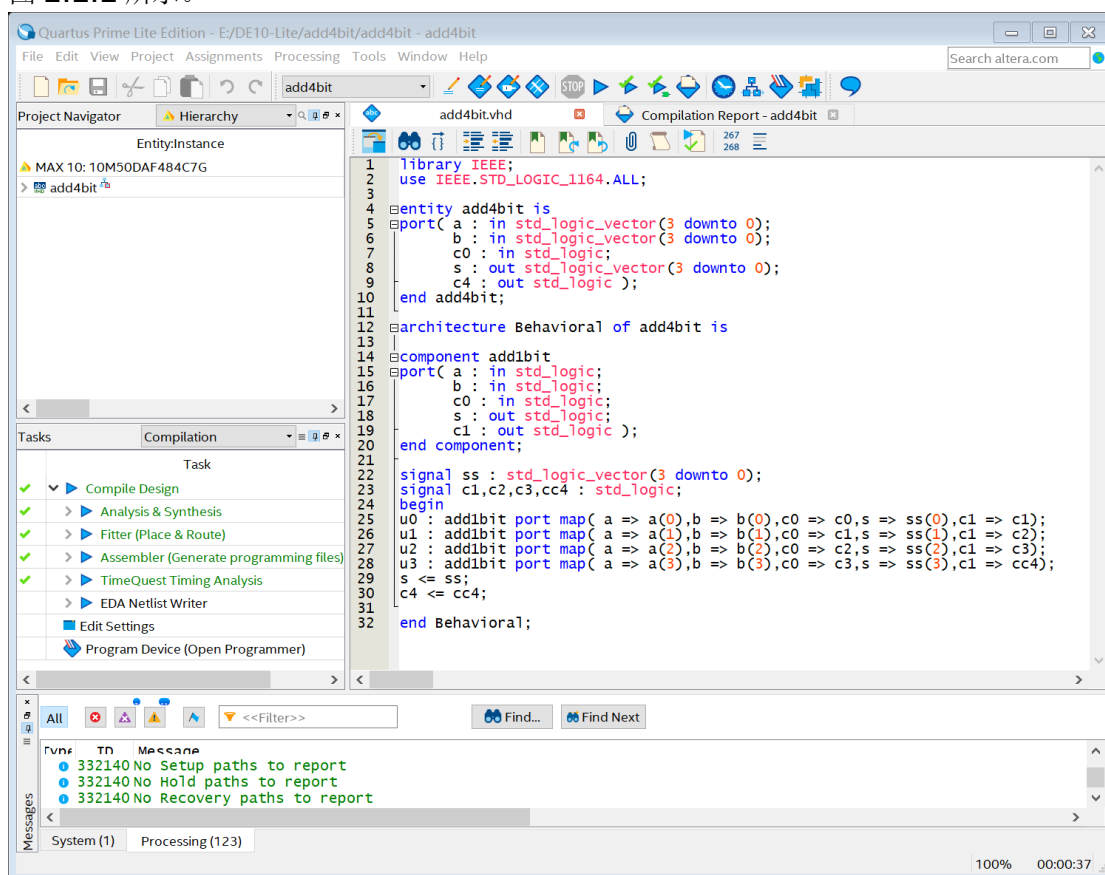
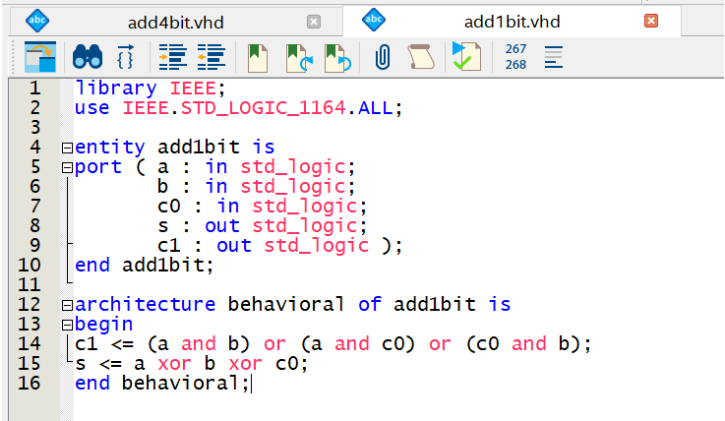


图 2.2.2 4 位全加器 add4bit 项目文件

在这个 4 位全加器的源程序中，定义了一个 1 位全加器(add1bit)的元件，然后通过“port map”指令将 1 位全加器的 5 个输入输出信号与 4 位全加器的信号一一对应地联系起来了。编写完 4 位全加器的顶层设计文件后，还得在这个项目文件夹中编写 1 位全加器 add1bit.vhd 的底层设计文件，如图 2.2.3 所示。为了使项目编译能正常进行，底层设计文件必须保存在与顶层设计文件相同的目录下。



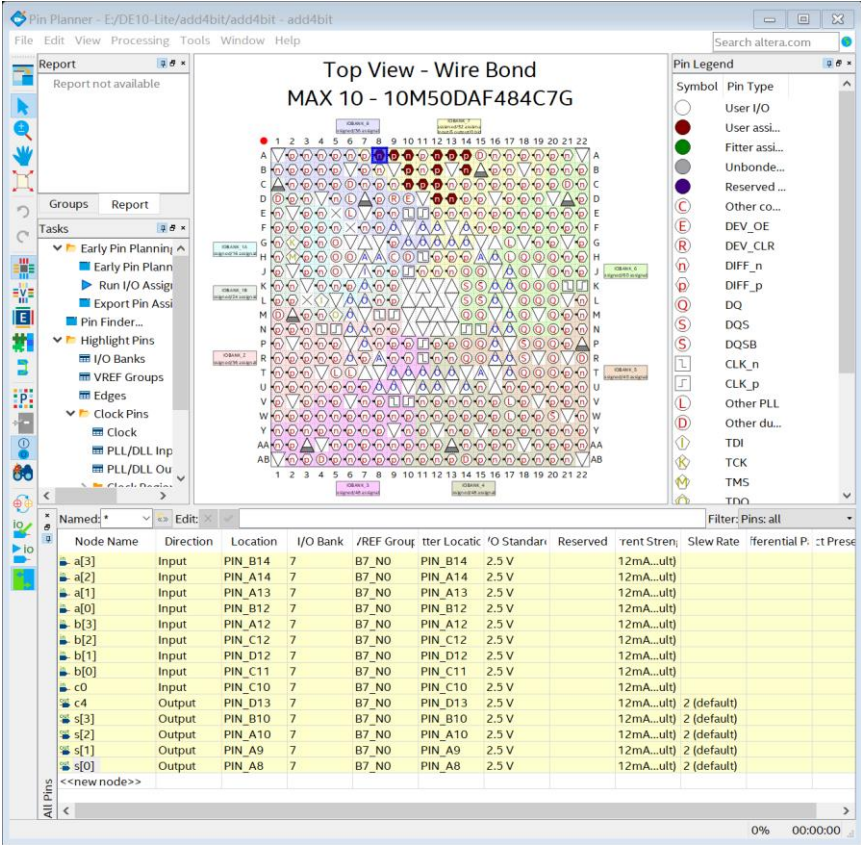
```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity add1bit is
5  port ( a : in std_logic;
6        b : in std_logic;
7        c0 : in std_logic;
8        s : out std_logic;
9        c1 : out std_logic );
10 end add1bit;
11
12 architecture behavioral of add1bit is
13 begin
14   c1 <= (a and b) or (a and c0) or (b and c0);
15   s <= a xor b xor c0;
16 end behavioral;

```

图 2.2.3 4 位全加器中的底层文件 add1bit.vhd

先对程序进行编译，然后执行 Assignments>Pin planner 菜单命令，进行引脚分配后的引脚分配编辑器窗口如图 2.2.4 所示。在这里将拨码开关 SW8、SW7、SW6、SW5 作为 a3、a2、a1、a0 信号输入，将拨码开关 SW4、SW3、SW2、SW1 作为 b3、b2、b1、b0 信号输入，将拨码开关 SW0 作为 c0 信号输入，将 LEDR3、LEDR2、LEDR1、LEDR0 作为 s3、s2、s1、s0 信号的输出，将 LEDR4 作为 c4 信号的输出。



Node Name	Direction	Location	I/O Bank	/REF Group	Iter Locat	'O Standan	Reserved	trent Stre	Slew Rate	fferential P	it Prese
a[3]	Input	PIN_B14	7	B7_NO	PIN_B14	2.5 V		12mA...ult			
a[2]	Input	PIN_A14	7	B7_NO	PIN_A14	2.5 V		12mA...ult			
a[1]	Input	PIN_A13	7	B7_NO	PIN_A13	2.5 V		12mA...ult			
a[0]	Input	PIN_B12	7	B7_NO	PIN_B12	2.5 V		12mA...ult			
b[3]	Input	PIN_A12	7	B7_NO	PIN_A12	2.5 V		12mA...ult			
b[2]	Input	PIN_C12	7	B7_NO	PIN_C12	2.5 V		12mA...ult			
b[1]	Input	PIN_D12	7	B7_NO	PIN_D12	2.5 V		12mA...ult			
b[0]	Input	PIN_C11	7	B7_NO	PIN_C11	2.5 V		12mA...ult			
c0	Input	PIN_C10	7	B7_NO	PIN_C10	2.5 V		12mA...ult			
c4	Output	PIN_D13	7	B7_NO	PIN_D13	2.5 V		12mA...ult	2 (default)		
s[3]	Output	PIN_B10	7	B7_NO	PIN_B10	2.5 V		12mA...ult	2 (default)		
s[2]	Output	PIN_A10	7	B7_NO	PIN_A10	2.5 V		12mA...ult	2 (default)		
s[1]	Output	PIN_A9	7	B7_NO	PIN_A9	2.5 V		12mA...ult	2 (default)		
s[0]	Output	PIN_A8	7	B7_NO	PIN_A8	2.5 V		12mA...ult	2 (default)		

图 2.2.29 4 位全加器引脚分配编辑器窗口

紧接着进行未用引脚的设置,执行 **Assignments>Device** 菜单命令,继续点击 **Device and Pin Options...** 按键, 出现 **Device and Pin Options** 设置窗口, 选中 **Unused Pins** 条目, 在 **Reserve all unused pins** 下拉列表中选择 **As input tri-stated** 设置项, 点击 **OK** 按键返回主界面。

重新对项目进行完整编译后生成 **add4bit.sof** 文件,经程序下载后就可以在 **DE10-Lite** 开发板上验证程序的正确与否了。

FPGA 部分实验三：时序电路

3.1 DE10-Lite 开发板时序电路设计实验- 1Hz 信号发生实验

在 DE10-Lite 开发板提供了两个 50MHz 时钟信号，该时钟信号连接到 FPGA 中作为用户逻辑时钟使用。DE10-Lite 开发板时钟分配框图如图 3.1.1 所示。

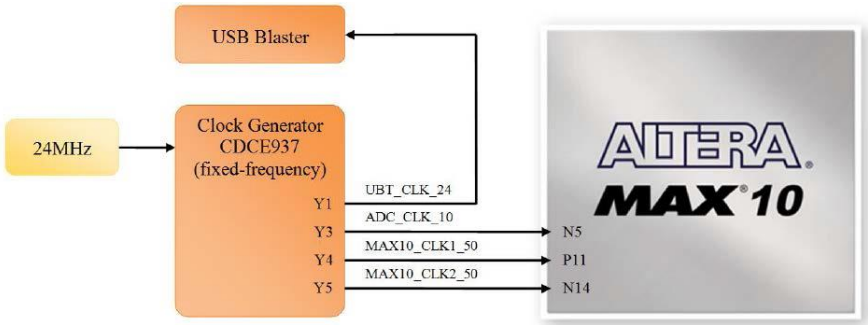


图 3.1.1 时钟分配框图

时钟输入到 FPGA 的 I/O 引脚的相关引脚分配如下表所示。

信号名称	FPGA引脚号
MAX10_CLK1_50	PIN P11
MAX10_CLK2_50	PIN N14
ADC_CLK_10	PIN N5

50MHz 方波信号连接到 FPGA 芯片（MAX10M50DAF484C7G）的 PIN-P11 和 PIN-N14 号引脚。50MHz 方波信号频率太高了，我们的眼睛没办法观察，为了实验的方便需要产生一个 1Hz 的方波信号。

双击桌面上“Quartus(Quartus Prime 17.1) Lite Edition”图标，启动 Quartus 软件(也可从开始菜单启动)。选择 File | New Project Wizard...菜单命令，在弹出的对话框中输入项目文件夹、项目名称和顶层实体名均为 clk1hz，如图 3.1.2 所示。选择芯片如图 3.1.3 所示。

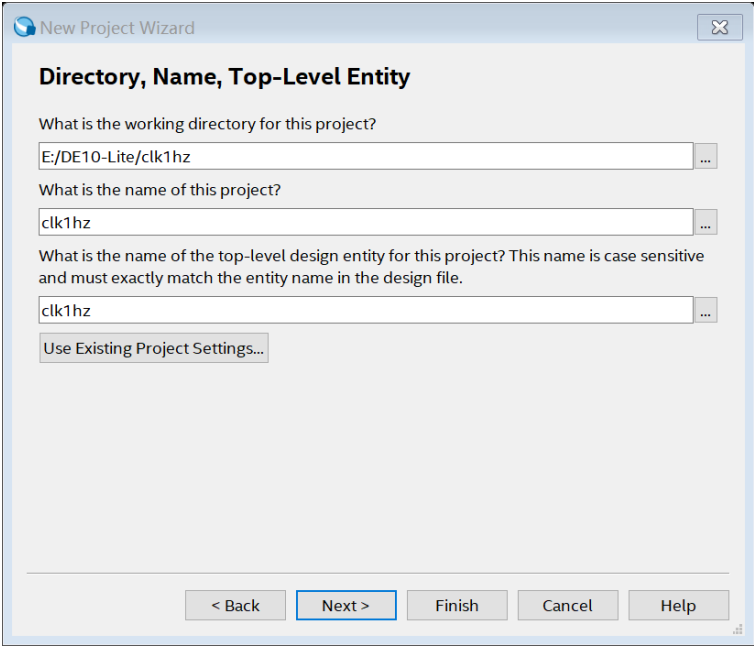


图 3.1.2 1Hz 方波信号项目 clk1hz

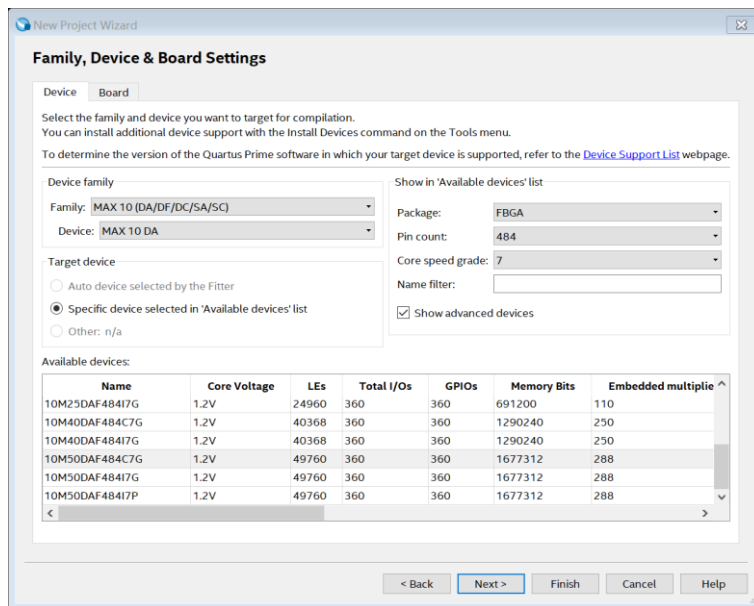


图 3.1.3 1Hz 方波信号 clk1hz 项目芯片选择

选择 File>New... 菜单命令，设计文件类型选择 VHDL File。进入到项目设计界面，输入 1Hz 方波信号 clk1hz 项目的 VHDL 源程序 clk1hz.vhd 后如图 3.1.4 所示。

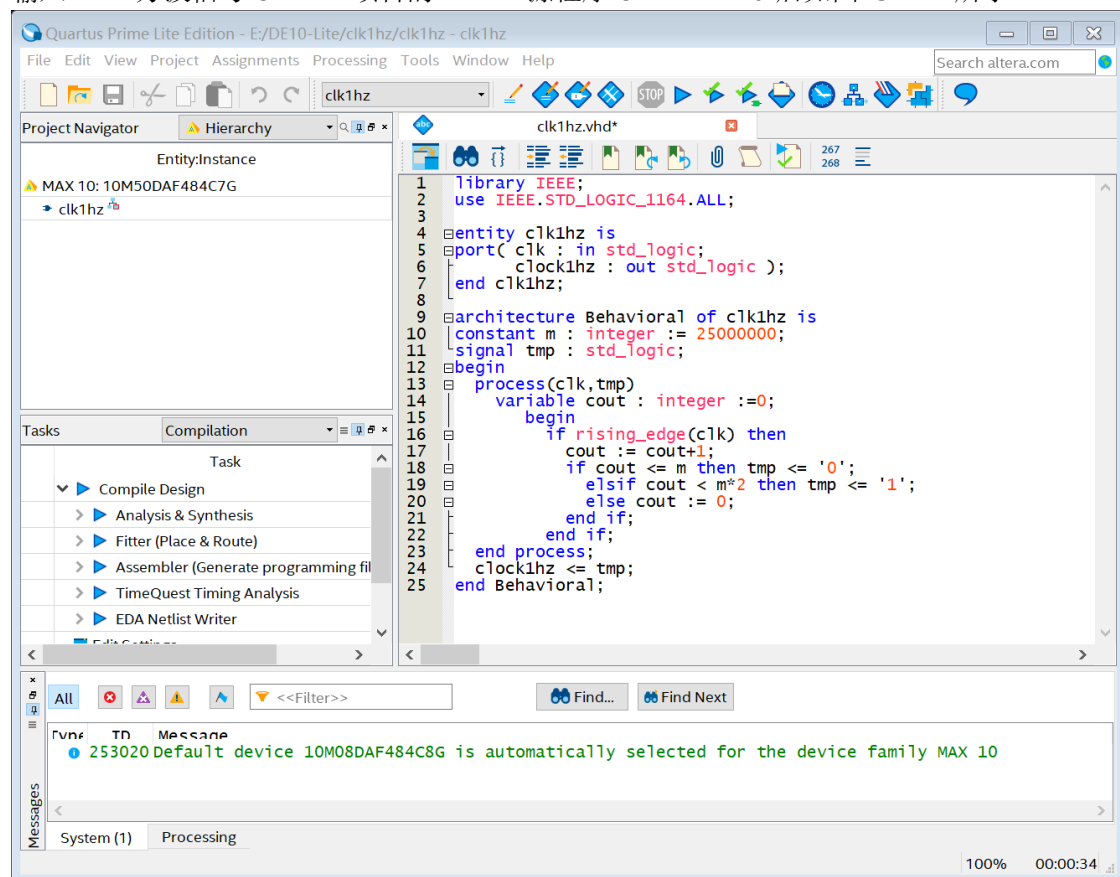


图 3.1.4 1Hz 方波信号 VHDL 源文件 clk1hz.vhd

在这个程序中，我们对 50MHz 的方波信号进行 50000000 分频，其中 25000000 个方波输出高电平，25000000 个方波输出低电平，最终得到 1Hz 方波信号。引脚分配如图 3.1.5 所示。clk 信号接芯片引脚 PIN-P11，输出信号接 LEDR0。

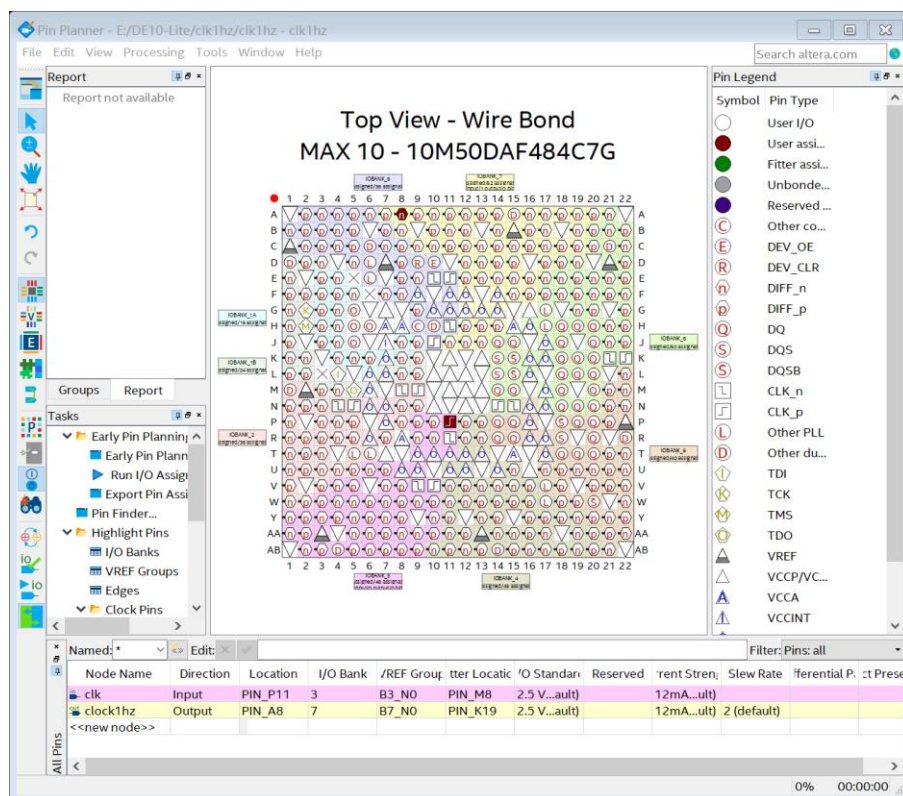


图 3.1.5 1Hz 方波信号引脚分配

紧接着进行未用引脚的设置,执行 **Assignments>Device** 菜单命令,继续点击 **Device and Pin Options...** 按键, 出现 **Device and Pin Options** 设置窗口, 选中 **Unused Pins** 条目, 在 **Reserve all unused pins** 下拉列表中选择 **As input tri-stated** 设置项, 点击 **OK** 按键返回主界面。

重新对项目进行完整编译后生成 **clk1hz.sof** 文件, 下载后就可以在 **DE10-Lite** 开发板上验证程序的正确与否了。

3.2 DE10-Lite 开发板时序电路设计实验 – 十进制计数器实验一

在产生了 1Hz 的信号后，我们就可以拿这个信号来做一些时序电路的实验了，首先可以做十进制计数器的实验。我们把这个十进制计数器的项目命名为 **countera**。项目的目录为 **E:\DE10-Lite\countera**。

由于前面已经做了多个实验，同学们对 Quartus Prime 17.1 集成开发软件的操作应该比较熟悉了，所以今后的实验我们只给出源程序了。十进制计数器的源程序如图 3.2.1 所示。

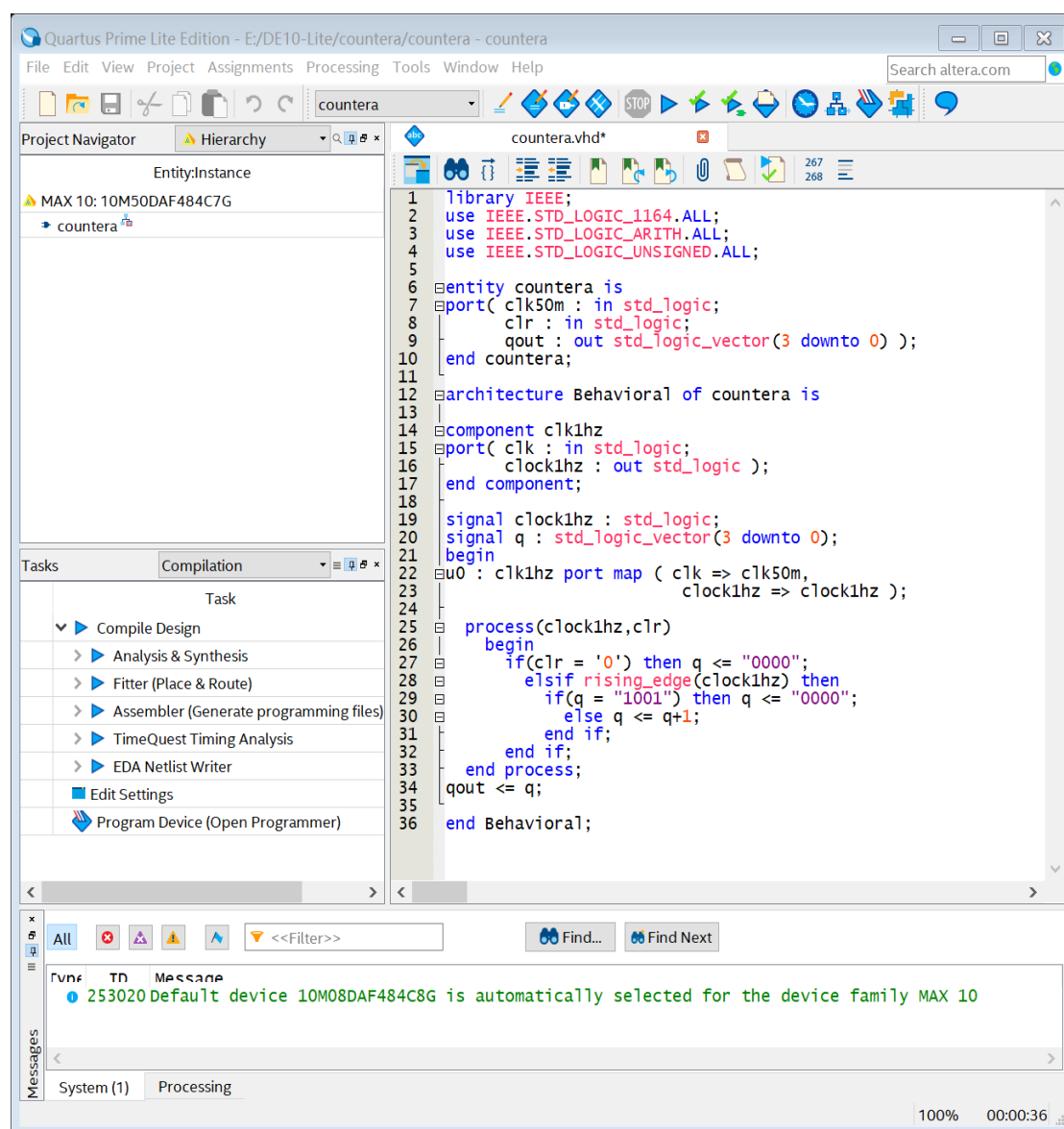


图 3.2.1 十进制计数器源程序

在这个十进制计数器的源程序中包含有 **component clk1hz**，所以应该把 **clk1hz.vhd** 这个文件拷贝到 **E:\DE10-Lite\countera** 目录下，Quartus Prime 在编译时会自动寻找到这个底层文件 **clk1hz.vhd** 的。引脚分配如图 3.2.2 所示，在这里将输出的十进制数 **qout3**、**qout2**、**qout1**、**qout0** 分配到 **LEDR3**、**LEDR2**、**LEDR1**、**LEDR0** 上，清零信号分配到按钮开关 **KEY0** 上。

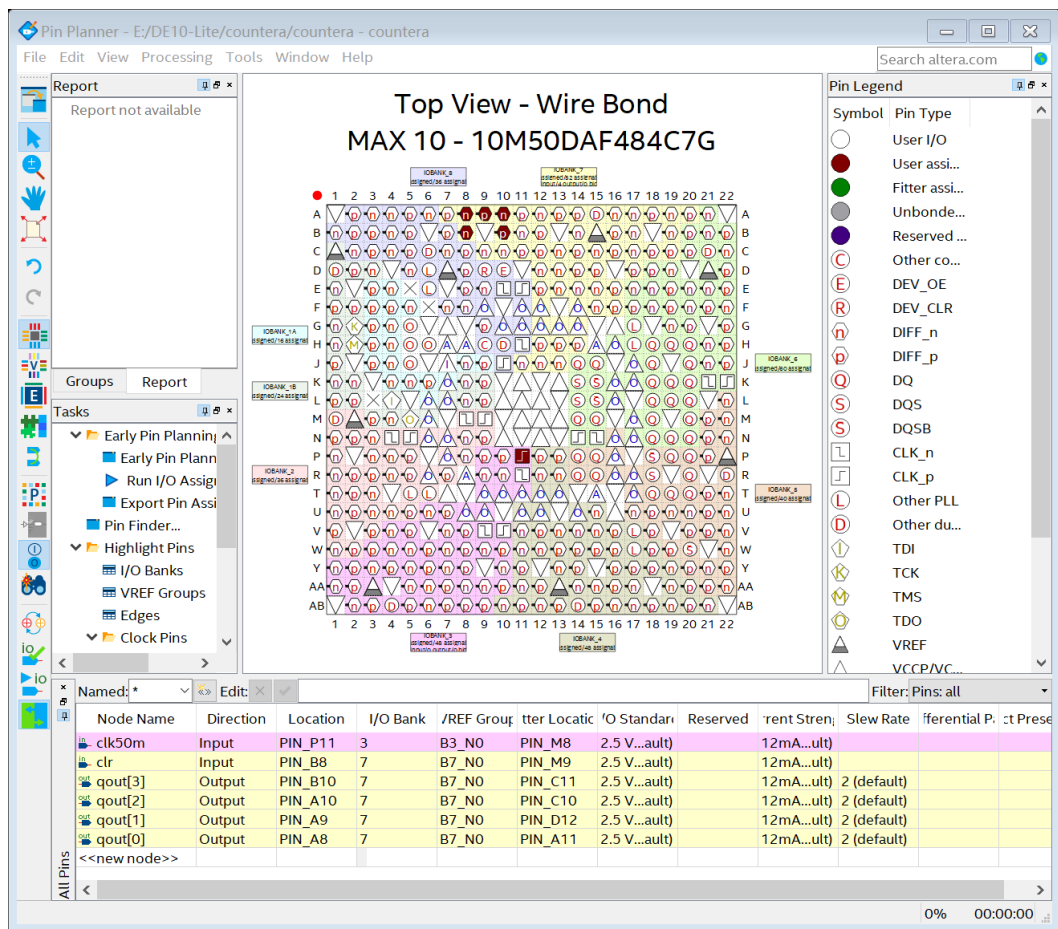


图 3.2.2 十进制计数器引脚分配

在引脚分配做完后，还是不要忘记对未用引脚的设置，最后需要对项目进行完全编译生产 **countera.sof** 文件，下载到 DE10-Lite 开发板验证设计是否正确。

3.3 DE10-Lite 开发板时序电路设计实验 – 七段数码管显示实验

在上一个实验中，我们虽然完成了十进制计数，但计数结果是在 4 个发光二极管上显示的，显得别扭。所以我们在这个实验中将学习 DE10-Lite 开发板上的数码管显示方法。DE10-Lite 开发板上数码管显示电路如图 3.3.1 所示，每个数码管内含 8 个发光二极管，通过点亮不同的发光二极管，即可显示相应的字符。

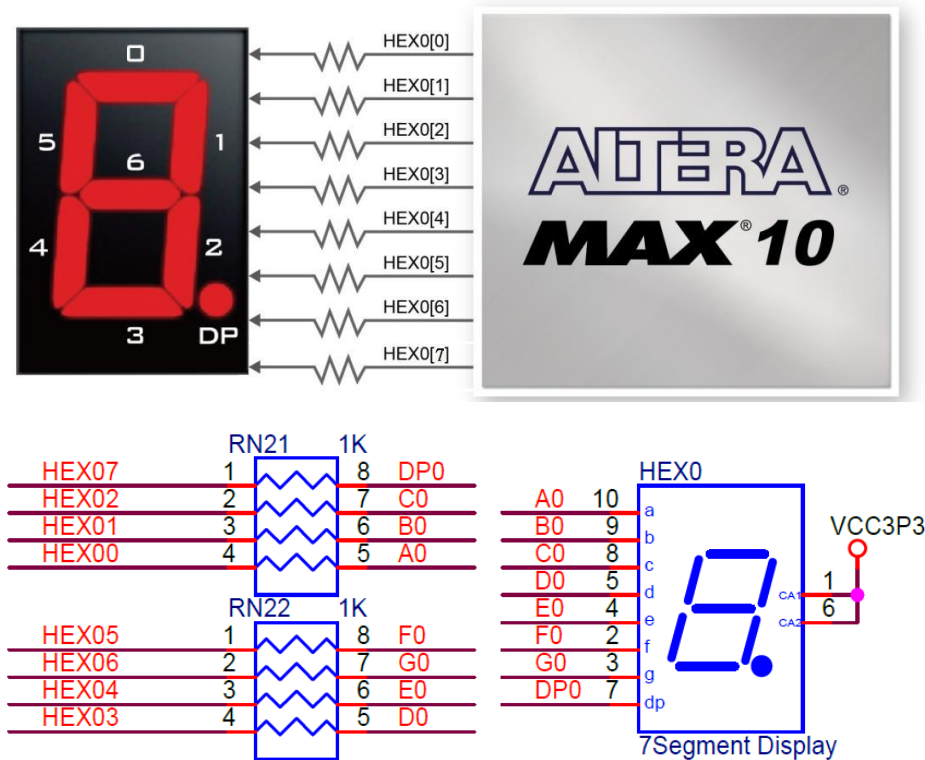


图 3.3.1 数码管显示电路

比如我们想在最右边的数码管 HEX0 上显示一个数字“0”，那么我们就需要点亮 HEX0 数码管上的 0、1、2、3、4、5 段的发光二极管。由于采用了共阳数码管，当 FPGA 对应输出端口为低电平时，点亮数码管相应的段；当 FPGA 对应输出端口为高电平时，熄灭数码管相应的段。在了解了数码管显示电路后，我们就可以写一个简单的程序来体会一下数码管显示的快乐了。我们把这个数码管显示的项目命名为 hex7seg。项目的目录为 E:\DE10-Lite\hex7seg。

数码管 HEX0 的连接到 FPGA 的引脚对照表如下表所示：

Signal Name	FPGA Pin No.	Description	I/O Standard
HEX00	PIN_C14	Seven Segment Digit 0[0]	3.3-V LVTTTL
HEX01	PIN_E15	Seven Segment Digit 0[1]	3.3-V LVTTTL
HEX02	PIN_C15	Seven Segment Digit 0[2]	3.3-V LVTTTL
HEX03	PIN_C16	Seven Segment Digit 0[3]	3.3-V LVTTTL
HEX04	PIN_E16	Seven Segment Digit 0[4]	3.3-V LVTTTL
HEX05	PIN_D17	Seven Segment Digit 0[5]	3.3-V LVTTTL
HEX06	PIN_C17	Seven Segment Digit 0[6]	3.3-V LVTTTL
HEX07	PIN_D15	Seven Segment Digit 0[7], DP	3.3-V LVTTTL

数码管显示的源程序如图 3.3.2 所示。我们通过开发板上的开关 SW3、SW2、SW1、SW0 来输入一个十六进制数，在数码管 HEX0 上显示这个十六进制数。

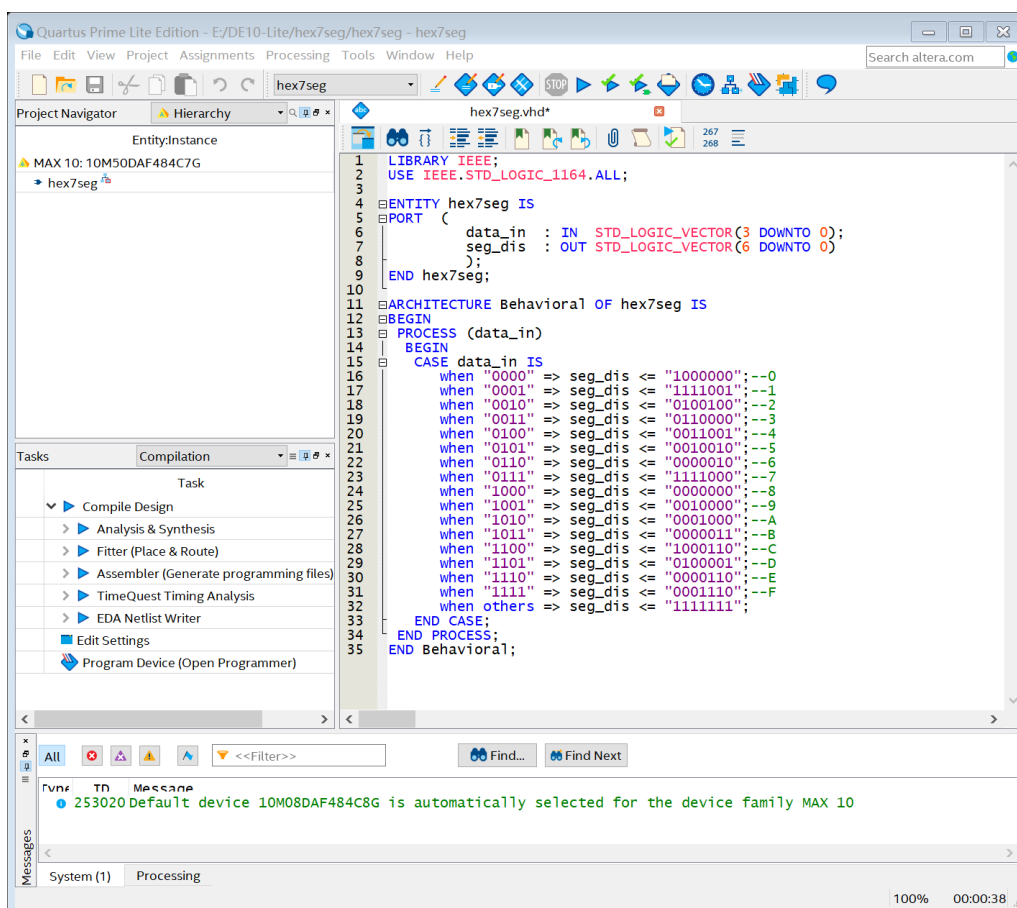


图 3.3.2 数码管显示源程序 hex7seg.vhd

引脚分配如图 3.3.3。

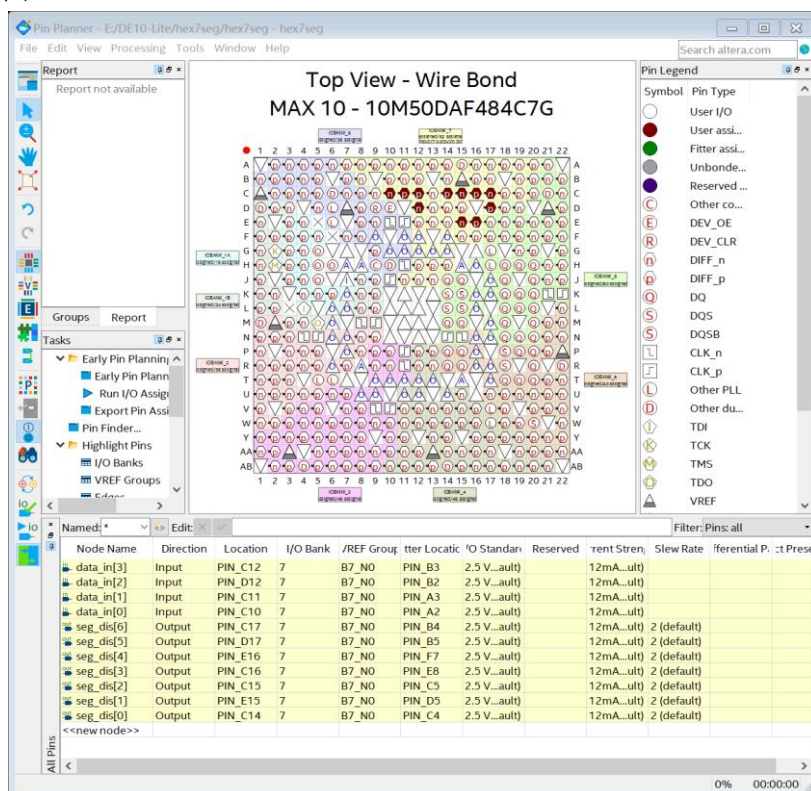


图 3.11 数码管显示引脚分配

3.4 DE10-Lite 开发板时序电路设计实验 – 十进制计数器实验二

在前面的十进制计数器实验一中，我们在发光二极管 LEDR3-LEDRO 上显示了一个十进制数。在这个实验中，我们将在数码管上显示计数器的计数值。我们把这个十进制计数器的项目命名为 counterb。项目的目录为 E:\DE10-Lite\counterb。十进制计数器的源程序如图 3.4.1 所示。

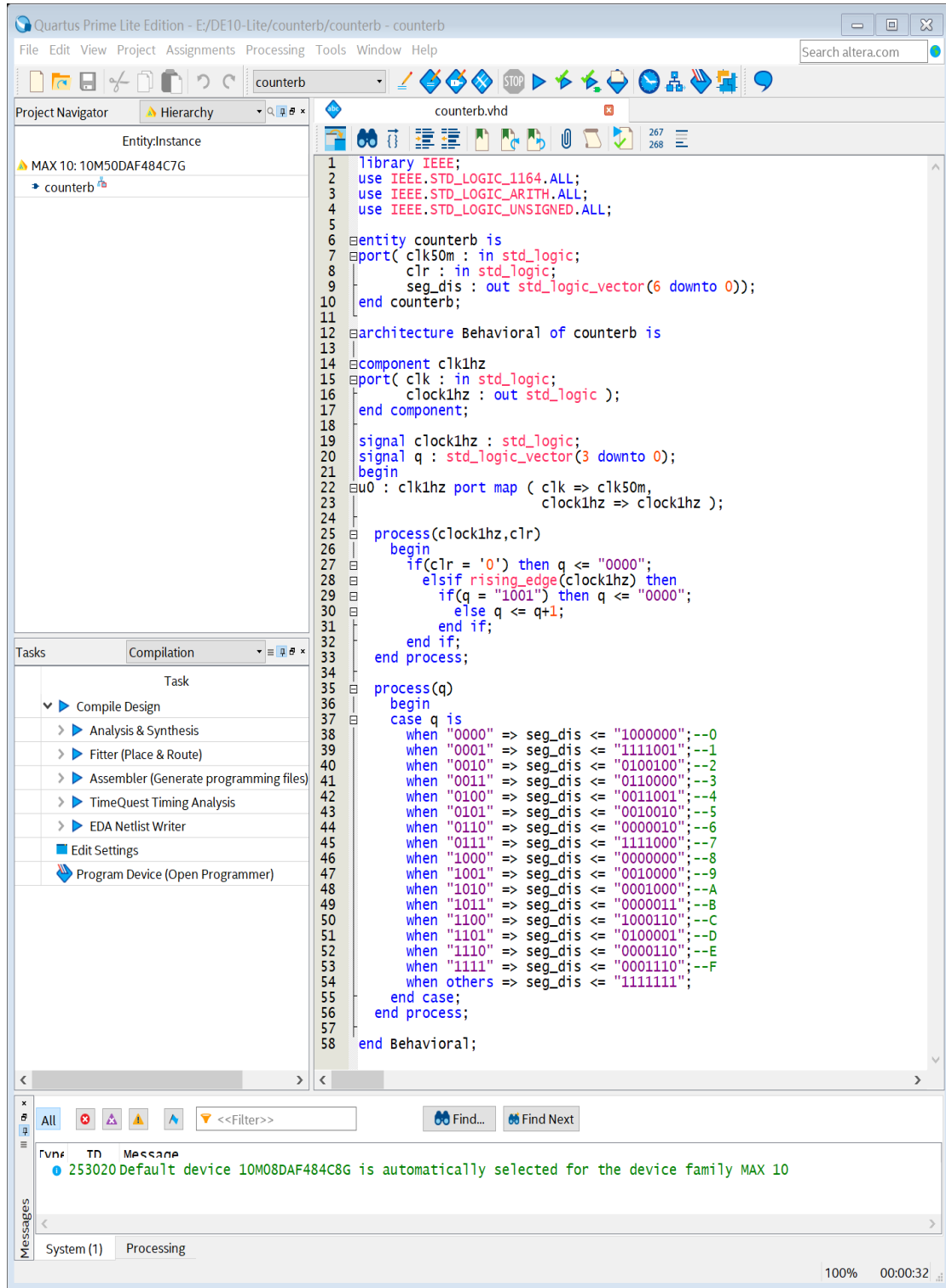


图 3.4.1 十进制计数器二源程序 counterb.vhd

由于在程序中把 1Hz 信号模块作为一个元件使用了,即把 clk1hz.vhd 作为底层设计,因此需要把 clk1hz.vhd 的源程序拷贝到本项目的文件夹下面来。

本项目的引脚分配如图 3.4.2 。

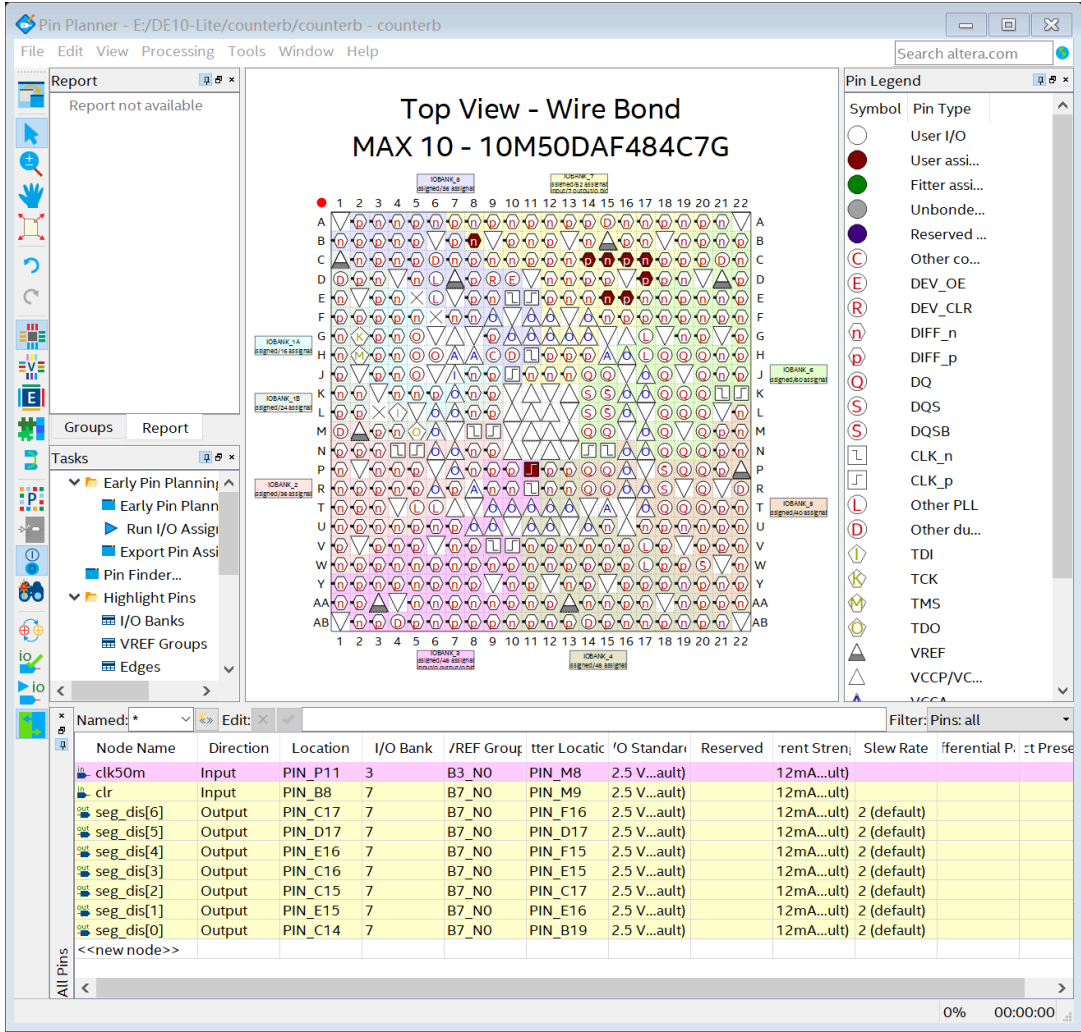


图 3.15 十进制计数器二引脚分配

在经过完全编译后,将 counterb.sof 下载到开发板中,就可以看到在数码管 HEX0 上显示十进制计数器的输出了,按下复位按键后可以复位计数值。

FPGA 部分实验四：综合设计

4.1 DE10-Lite 开发板综合设计实验 – 数字钟实验一

在完成了时序电路设计实验后，我们对 FPGA 的设计有了一定的认识，我们可以做一些有针对性的项目了，在这一节里以数字钟为例，来体会一下完成一个项目的全过程。

数字钟是我们非常熟悉的一个电子产品，分为秒、分钟、小时三个计量单位，前 2 个是 60 进制，小时是 24 进制，可以用 6 个数码管显示。在我们这块 DE10-Lite 开发板上，刚好有 6 个数码管，因此我们可以用这 6 个数码管做一个 24 小时计时的数字钟。

我们把这个数字钟的项目命名为 clocka。项目的 VHDL 程序如图 4.1.1 所示。

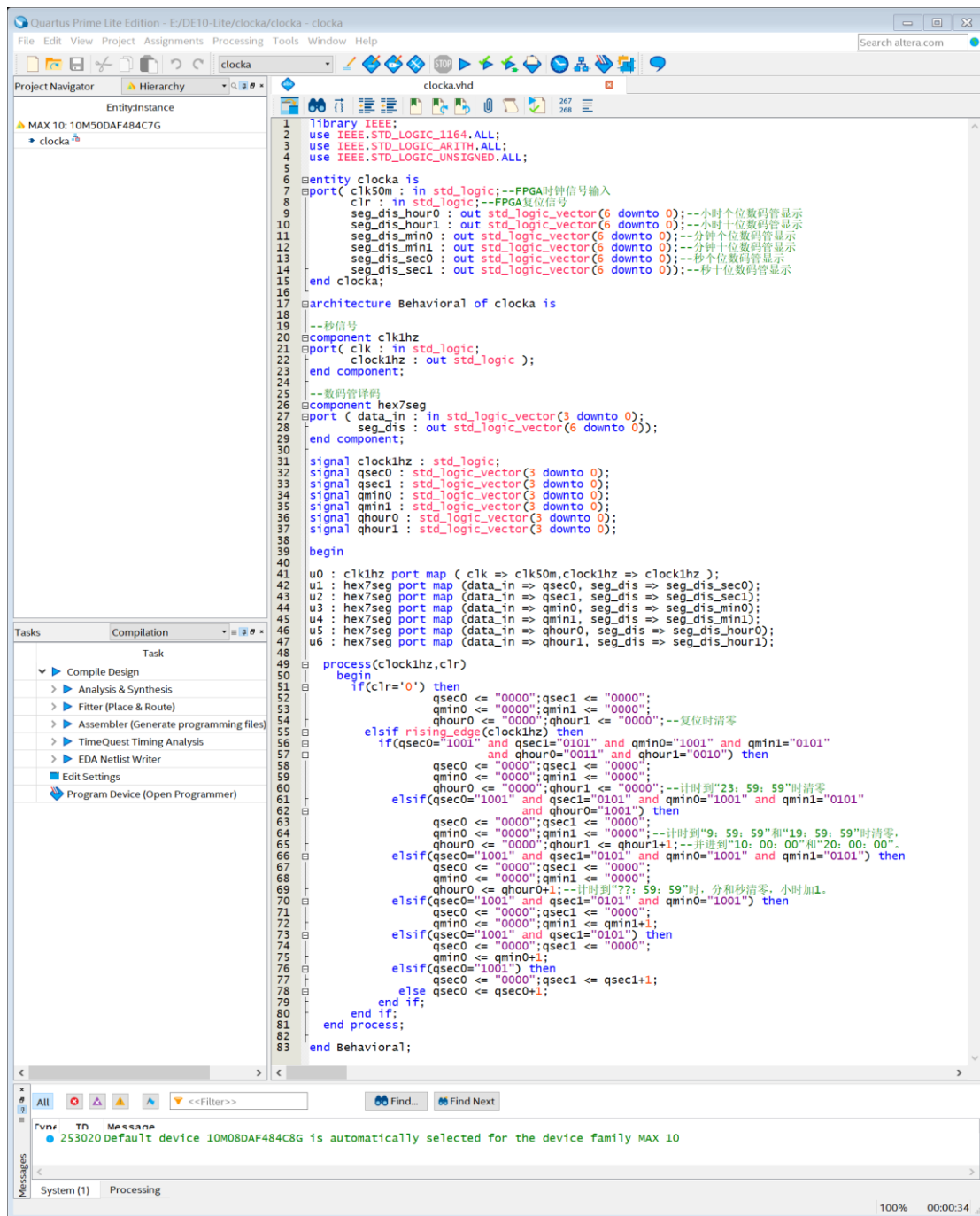
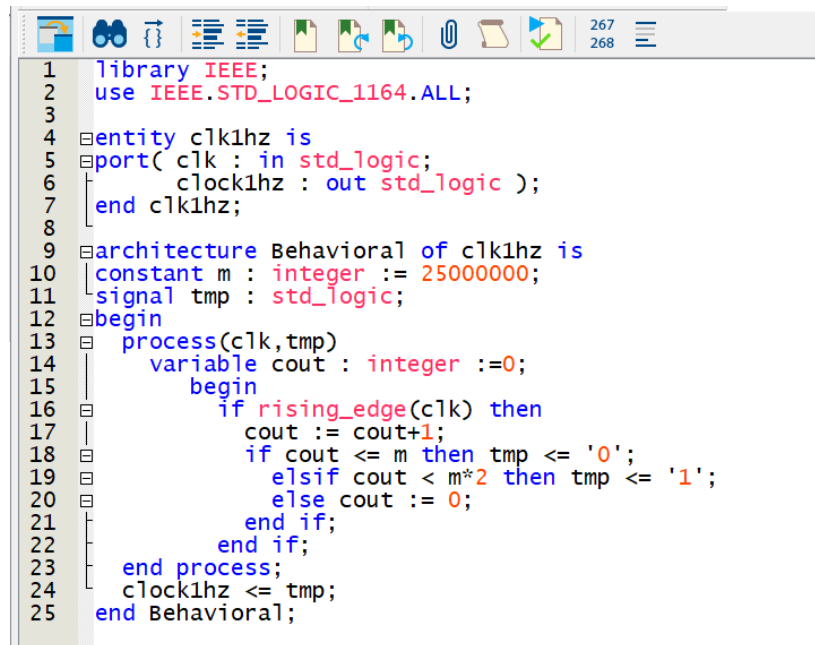


图 4.1.1 数字钟 clocka 顶层 VHDL 程序

在数字钟 clocka 顶层 VHDL 程序，包含有 7 个底层程序（component），u0-clk1hz 程序将开发板上的 50MHz 时钟信号分频后产生 1Hz 的信号，用于数字钟的秒信号计数。u1 – u6 这 6 个底层程序 hex7seg.vhd 用于将计数值译码为七段显示码，可以将数字钟的计数值在 6 个数码管上显示。在顶层程序 clocka.vhd 编译前，需要将 clk1hz.vhd 和 hex7seg.vhd 两个文件拷贝到当前文件夹中。

图 4.1.2、图 4.1.3 分别是 clk1hz.vhd 和 hex7seg.vhd 的 VHDL 源程序。

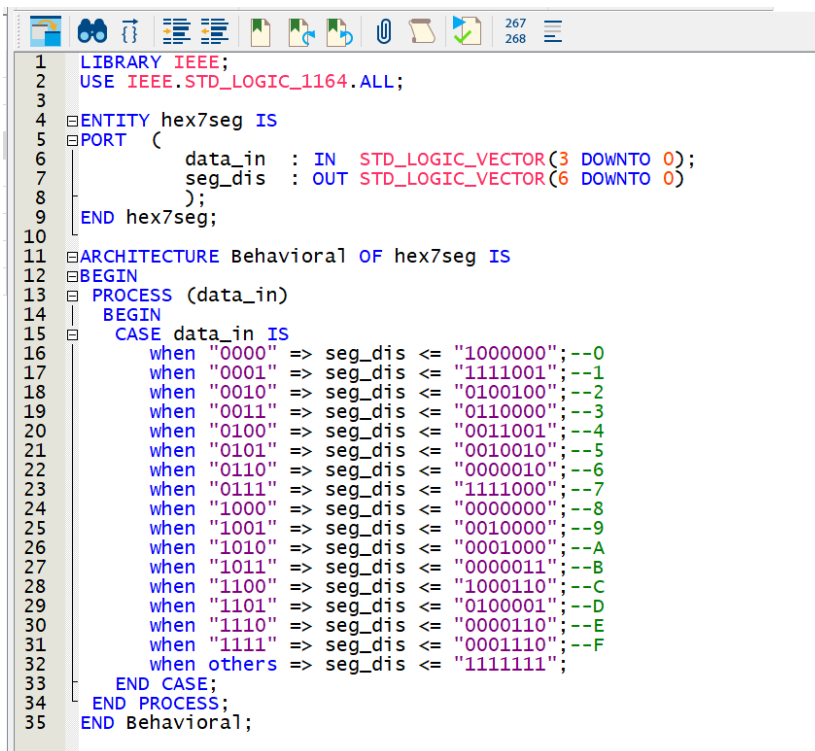


```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity clk1hz is
5  port( clk : in std_logic;
6        clock1hz : out std_logic );
7  end clk1hz;
8
9  architecture Behavioral of clk1hz is
10 | constant m : integer := 25000000;
11 | signal tmp : std_logic;
12 | begin
13 |   process(clk,tmp)
14 |   variable cout : integer :=0;
15 |   begin
16 |     if rising_edge(clk) then
17 |       cout := cout+1;
18 |       if cout <= m then tmp <= '0';
19 |       elsif cout < m*2 then tmp <= '1';
20 |       else cout := 0;
21 |       end if;
22 |     end if;
23 |   end process;
24 |   clock1hz <= tmp;
25 | end Behavioral;

```

图 4.1.2 1Hz 信号产生的元件模块源程序



```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY hex7seg IS
5  PORT (
6      data_in : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
7      seg_dis : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)
8  );
9  END hex7seg;
10
11 ARCHITECTURE Behavioral OF hex7seg IS
12 BEGIN
13   PROCESS (data_in)
14   BEGIN
15     CASE data_in IS
16     when "0000" => seg_dis <= "1000000";--0
17     when "0001" => seg_dis <= "1111001";--1
18     when "0010" => seg_dis <= "0100100";--2
19     when "0011" => seg_dis <= "0110000";--3
20     when "0100" => seg_dis <= "0011001";--4
21     when "0101" => seg_dis <= "0010010";--5
22     when "0110" => seg_dis <= "0000010";--6
23     when "0111" => seg_dis <= "1111000";--7
24     when "1000" => seg_dis <= "0000000";--8
25     when "1001" => seg_dis <= "0010000";--9
26     when "1010" => seg_dis <= "0001000";--A
27     when "1011" => seg_dis <= "0000011";--B
28     when "1100" => seg_dis <= "1000110";--C
29     when "1101" => seg_dis <= "0100001";--D
30     when "1110" => seg_dis <= "0000110";--E
31     when "1111" => seg_dis <= "0001110";--F
32     when others => seg_dis <= "1111111";
33   END CASE;
34   END PROCESS;
35 END Behavioral;

```

图 4.1.3 七段译码显示的元件模块源程序

数字钟实验用到了 6 个数码管，每个数码管需要 7 个引脚驱动，6 个数码管与 FPGA 芯片引脚对照表如下所示。

信号名称	FPGA 引脚号	信号名称	FPGA 引脚号	信号名称	FPGA 引脚号
HEX0-0	PIN_C14	HEX1-0	PIN_C18	HEX2-0	PIN_B20
HEX0-1	PIN_E15	HEX1-1	PIN_D18	HEX2-1	PIN_A20
HEX0-2	PIN_C15	HEX1-2	PIN_E18	HEX2-2	PIN_B19
HEX0-3	PIN_C16	HEX1-3	PIN_B16	HEX2-3	PIN_A21
HEX0-4	PIN_E16	HEX1-4	PIN_A17	HEX2-4	PIN_B21
HEX0-5	PIN_D17	HEX1-5	PIN_A18	HEX2-5	PIN_C22
HEX0-6	PIN_C17	HEX1-6	PIN_B17	HEX2-6	PIN_B22
HEX0-7	PIN_D15	HEX1-7	PIN_A16	HEX2-7	PIN_A19

信号名称	FPGA 引脚号	信号名称	FPGA 引脚号	信号名称	FPGA 引脚号
HEX3-0	PIN_F21	HEX4-0	PIN_F18	HEX5-0	PIN_J20
HEX3-1	PIN_E22	HEX4-1	PIN_E20	HEX5-1	PIN_K20
HEX3-2	PIN_E21	HEX4-2	PIN_E19	HEX5-2	PIN_L18
HEX3-3	PIN_C19	HEX4-3	PIN_J18	HEX5-3	PIN_N18
HEX3-4	PIN_C20	HEX4-4	PIN_H19	HEX5-4	PIN_M20
HEX3-5	PIN_D19	HEX4-5	PIN_F19	HEX5-5	PIN_N19
HEX3-6	PIN_E17	HEX4-6	PIN_F20	HEX5-6	PIN_N20
HEX3-7	PIN_D22	HEX4-7	PIN_F17	HEX5-7	PIN_L19

引脚分配如图 4.1.4 所示：

Named: * Edit:										Filter: Pins: all	
Node Name	Direction	Location	I/O Bank	/REF Group	Port Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	
clk50m	Input	PIN_P11	3	B3_NO	PIN_M8	2.5 V...alt)		12mA...ult)			
clr	Input	PIN_B8	7	B7_NO	PIN_M9	2.5 V...alt)		12mA...ult)			
seg_dis_hour0[6]	Output	PIN_F20	6	B6_NO	PIN_A14	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour0[5]	Output	PIN_F19	6	B6_NO	PIN_C16	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour0[4]	Output	PIN_H19	6	B6_NO	PIN_A13	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour0[3]	Output	PIN_J18	6	B6_NO	PIN_A15	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour0[2]	Output	PIN_E19	6	B6_NO	PIN_C14	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour0[1]	Output	PIN_E20	6	B6_NO	PIN_J12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour0[0]	Output	PIN_F18	6	B6_NO	PIN_B15	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[6]	Output	PIN_N20	6	B6_NO	PIN_D12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[5]	Output	PIN_N19	6	B6_NO	PIN_J13	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[4]	Output	PIN_M20	6	B6_NO	PIN_E12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[3]	Output	PIN_N18	6	B6_NO	PIN_D13	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[2]	Output	PIN_L18	6	B6_NO	PIN_H13	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[1]	Output	PIN_K20	6	B6_NO	PIN_H14	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_hour1[0]	Output	PIN_J20	6	B6_NO	PIN_A11	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[6]	Output	PIN_B22	6	B6_NO	PIN_H12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[5]	Output	PIN_C22	6	B6_NO	PIN_J11	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[4]	Output	PIN_B21	6	B6_NO	PIN_E11	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[3]	Output	PIN_A21	6	B6_NO	PIN_B12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[2]	Output	PIN_B19	7	B7_NO	PIN_B14	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[1]	Output	PIN_A20	7	B7_NO	PIN_D11	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min0[0]	Output	PIN_B20	6	B6_NO	PIN_A7	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[6]	Output	PIN_E17	6	B6_NO	PIN_E10	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[5]	Output	PIN_D19	6	B6_NO	PIN_A9	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[4]	Output	PIN_C20	6	B6_NO	PIN_C8	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[3]	Output	PIN_C19	7	B7_NO	PIN_B10	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[2]	Output	PIN_E21	6	B6_NO	PIN_B8	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[1]	Output	PIN_E22	6	B6_NO	PIN_C9	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_min1[0]	Output	PIN_F21	6	B6_NO	PIN_H11	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[6]	Output	PIN_C17	7	B7_NO	PIN_A17	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[5]	Output	PIN_D17	7	B7_NO	PIN_D15	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[4]	Output	PIN_E16	7	B7_NO	PIN_B16	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[3]	Output	PIN_C16	7	B7_NO	PIN_E13	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[2]	Output	PIN_C15	7	B7_NO	PIN_A16	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[1]	Output	PIN_E15	7	B7_NO	PIN_D14	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec0[0]	Output	PIN_C14	7	B7_NO	PIN_C15	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[6]	Output	PIN_B17	7	B7_NO	PIN_A10	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[5]	Output	PIN_A18	7	B7_NO	PIN_C12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[4]	Output	PIN_A17	7	B7_NO	PIN_A8	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[3]	Output	PIN_B16	7	B7_NO	PIN_C13	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[2]	Output	PIN_E18	6	B6_NO	PIN_A12	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[1]	Output	PIN_D18	6	B6_NO	PIN_C10	2.5 V...alt)		12mA...ult)	2 (default)		
seg_dis_sec1[0]	Output	PIN_C18	7	B7_NO	PIN_C11	2.5 V...alt)		12mA...ult)	2 (default)		
<<new node>>											

图 4.1.4 数字钟 clocka 引脚分配

在经过完全编译后我们把 clocka.sof 文件通过下载器下载后，在 DE10-Lite 开发板上就可以看到数字钟开始计时了。

4.2 DE10-Lite 开发板综合设计实验 – 数字钟实验二

上一个实验我们做了一个数字钟，但这个数字钟有个问题，就是每次运行后都是从零点开始计时，这样的数字钟是没人买的，我们至少还得加入时间调整功能。

利用 DE10-Lite 开发板上的按键作为时间调整键，分别是：SW3 键作为小时增加键、SW2 键作为小时减小键、SW1 键作为分钟增加键、SW0 键作为分钟减小键。其它的设置和要求与数字钟实验一相同。我们把这个数字钟的项目命名为 clockb。项目的 VHDL 程序如图 4.2.1 所示。这个程序与 clocka 项目的顶层设计差不多，同学们可以在 clocka 的基础上修改好了。

```
clockb.vhd
Compilation Report - clockb
267
268

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clockb is
7  port( clk50m : in std_logic; --FPGA时钟信号输入
8        clr : in std_logic; --FPGA复位信号
9        inc_min : in std_logic; --分调整增加
10       dec_min : in std_logic; --分调整减小
11       inc_hour : in std_logic; --小时调整增加
12       dec_hour : in std_logic; --小时调整减小
13       seg_dis_hour0 : out std_logic_vector(6 downto 0); --小时个位数码管显示
14       seg_dis_hour1 : out std_logic_vector(6 downto 0); --小时十位数码管显示
15       seg_dis_min0 : out std_logic_vector(6 downto 0); --分钟个位数码管显示
16       seg_dis_min1 : out std_logic_vector(6 downto 0); --分钟十位数码管显示
17       seg_dis_sec0 : out std_logic_vector(6 downto 0); --秒个位数码管显示
18       seg_dis_sec1 : out std_logic_vector(6 downto 0); --秒十位数码管显示
19  end clockb;
20
21  architecture Behavioral of clockb is
22  --秒信号
23  component clk1hz
24  port( clk : in std_logic;
25        clock1hz : out std_logic );
26  end component;
27
28  --数码管译码
29  component hex7seg
30  port ( data_in : in std_logic_vector(3 downto 0);
31        seg_dis : out std_logic_vector(6 downto 0));
32  end component;
33
34  signal clock1hz : std_logic;
35  signal qsec0 : std_logic_vector(3 downto 0);
36  signal qsec1 : std_logic_vector(3 downto 0);
37  signal qmin0 : std_logic_vector(3 downto 0);
38  signal qmin1 : std_logic_vector(3 downto 0);
39  signal qhour0 : std_logic_vector(3 downto 0);
40  signal qhour1 : std_logic_vector(3 downto 0);
41
42  begin
43
44  u0 : clk1hz port map ( clk => clk50m, clock1hz => clock1hz );
45  u1 : hex7seg port map (data_in => qsec0, seg_dis => seg_dis_sec0);
46  u2 : hex7seg port map (data_in => qsec1, seg_dis => seg_dis_sec1);
47  u3 : hex7seg port map (data_in => qmin0, seg_dis => seg_dis_min0);
48  u4 : hex7seg port map (data_in => qmin1, seg_dis => seg_dis_min1);
49  u5 : hex7seg port map (data_in => qhour0, seg_dis => seg_dis_hour0);
50  u6 : hex7seg port map (data_in => qhour1, seg_dis => seg_dis_hour1);
51
52  process(clock1hz, clr)
53  begin
54      if(clr='0') then
55          qsec0 <= "0000"; qsec1 <= "0000";
56          qmin0 <= "0000"; qmin1 <= "0000";
57          qhour0 <= "0000"; qhour1 <= "0000"; --复位时清零
58      elsif rising_edge(clock1hz) then
59          --分调整增加
60          if(inc_min='1') then
61              if(qmin0="1001" and qmin1="0101") then
62                  qmin0 <= "0000"; qmin1 <= "0000"; --59分时归零
63              elsif(qmin0="1001") then
64                  qmin0 <= "0000"; qmin1 <= qmin1+1; --分个位为9时进位
65              else qmin0 <= qmin0+1;
66          end if;
67          --分调整减小
68          elsif(dec_min='1') then
69              if(qmin0="0000" and qmin1="0000") then
70                  qmin0 <= "1001"; qmin1 <= "0101"; --0分时变为59分
71              elsif(qmin0="0000") then
72                  qmin0 <= "1001"; qmin1 <= qmin1-1; --分个位为0时借位
73              else qmin0 <= qmin0-1;
74          end if;
75      end if;
```

```

76  --小时调整增加
77  elsif(inc_hour='1') then
78      if(qhour0="0011" and qhour1="0010") then
79          qhour0 <= "0000"; qhour1 <= "0000";
80      elsif(qhour0="1001") then
81          qhour0 <= "0000"; qhour1 <= qhour1+1;
82      else qhour0 <= qhour0+1;
83      end if;
84  --小时调整减小
85  elsif(dec_hour='1') then
86      if(qhour0="0000" and qhour1="0000") then
87          qhour0 <= "0011"; qhour1 <= "0010";
88      elsif(qhour0="0000") then
89          qhour0 <= "1001"; qhour1 <= qhour1-1;
90      else qhour0 <= qhour0-1;
91      end if;
92
93      elsif(qsec0="1001" and qsec1="0101" and qmin0="1001" and qmin1="0101"
94          and qhour0="0011" and qhour1="0010") then
95          qsec0 <= "0000"; qsec1 <= "0000";
96          qmin0 <= "0000"; qmin1 <= "0000";
97          qhour0 <= "0000"; qhour1 <= "0000"; --计时到“23: 59: 59”时清零
98      elsif(qsec0="1001" and qsec1="0101" and qmin0="1001" and qmin1="0101"
99          and qhour0="1001") then
100          qsec0 <= "0000"; qsec1 <= "0000";
101          qmin0 <= "0000"; qmin1 <= "0000"; --计时到“9: 59: 59”和“19: 59: 59”时清零,
102          qhour0 <= "0000"; qhour1 <= qhour1+1; --并进到“10: 00: 00”和“20: 00: 00”。
103      elsif(qsec0="1001" and qsec1="0101" and qmin0="1001" and qmin1="0101") then
104          qsec0 <= "0000"; qsec1 <= "0000";
105          qmin0 <= "0000"; qmin1 <= "0000";
106          qhour0 <= qhour0+1; --计时到“?: 59: 59”时, 分和秒清零, 小时加1。
107      elsif(qsec0="1001" and qsec1="0101" and qmin0="1001") then
108          qsec0 <= "0000"; qsec1 <= "0000";
109          qmin0 <= "0000"; qmin1 <= qmin1+1;
110      elsif(qsec0="1001" and qsec1="0101") then
111          qsec0 <= "0000"; qsec1 <= "0000";
112          qmin0 <= qmin0+1;
113      elsif(qsec0="1001") then
114          qsec0 <= "0000"; qsec1 <= qsec1+1;
115      else qsec0 <= qsec0+1;
116      end if;
117  end if;
118  end process;
119
120  end Behavioral;

```

图 4.2.1 数字钟 clockb 顶层 VHDL 程序

在顶层程序 clocka.vhd 编译前, 需要将 clk1hz.vhd 和 hex7seg.vhd 两个文件拷贝到当前文件夹中。

与 clocka 相比, clockb 的引脚增加了 4 个, 引脚分配如图 4.2.2 所示。在经过完全编译后我们把 clockb.sof 文件通过 USB Blaster 下载后, 在 DE10-Lite 开发板上就可以看到数字钟开始计时了。现在我们可以调整时间了, 按 SW3 键和 SW2 键可以增加或减小小时数, 按 SW1 键和 SW0 键可以增加或减小分钟数, 将小时和分钟调整到当前的北京时间, 一个数字钟可以正常地运行了。

Pin Planner - E:/DE10-Lite/clockb/clockb - clockb

File Edit View Processing Tools Window Help

Search altera.com

Report Report not available

Groups Report

Tasks

- Early Pin Planning
 - Early Pin Plann
 - Run I/O Assign
 - Export Pin Assi
- Highlight Pins
 - I/O Banks

Top View - Wire Bond

MAX 10 - 10M50DAF484C7G

Pin Legend

Symbol	Pin Type
○	User I/O
●	User assi...
●	Fitter assi...
●	Unbonde...
●	Reserved ...
○	Other co...
○	DEV_OE
○	DEV_CLR
○	DIFF_n
○	DIFF_p
○	DQ
○	DQS
○	DQSB
○	CLK_n
○	CLK_p
○	Other PLL
○	Other du...
○	TDI
○	TCK
○	TMS

Node Name	Direction	Location	I/O Bank	/REF Group	ttter Locatic	/O Standari	Reserved	rent Streni	Slew Rate	fferential P	ct
clk50m	Input	PIN_P11	3	B3_NO	PIN_M8	2.5 V...ault		12mA...ult)			
clr	Input	PIN_B8	7	B7_NO	PIN_M9	2.5 V...ault		12mA...ult)			
dec_hour	Input	PIN_D12	7	B7_NO	PIN_C12	2.5 V...ault		12mA...ult)			
dec_min	Input	PIN_C10	7	B7_NO	PIN_A16	2.5 V...ault		12mA...ult)			
inc_hour	Input	PIN_C12	7	B7_NO	PIN_C15	2.5 V...ault		12mA...ult)			
inc_min	Input	PIN_C11	7	B7_NO	PIN_H14	2.5 V...ault		12mA...ult)			
seg_dis_hour0[6]	Output	PIN_F20	6	B6_NO	PIN_B19	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour0[5]	Output	PIN_F19	6	B6_NO	PIN_A18	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour0[4]	Output	PIN_H19	6	B6_NO	PIN_C9	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour0[3]	Output	PIN_J18	6	B6_NO	PIN_A8	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour0[2]	Output	PIN_E19	6	B6_NO	PIN_C14	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour0[1]	Output	PIN_E20	6	B6_NO	PIN_B10	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour0[0]	Output	PIN_F18	6	B6_NO	PIN_E14	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[6]	Output	PIN_N20	6	B6_NO	PIN_A14	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[5]	Output	PIN_N19	6	B6_NO	PIN_C16	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[4]	Output	PIN_M20	6	B6_NO	PIN_B14	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[3]	Output	PIN_N18	6	B6_NO	PIN_A15	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[2]	Output	PIN_L18	6	B6_NO	PIN_B15	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[1]	Output	PIN_K20	6	B6_NO	PIN_E13	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_hour1[0]	Output	PIN_J20	6	B6_NO	PIN_C13	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[6]	Output	PIN_B22	6	B6_NO	PIN_H13	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[5]	Output	PIN_C22	6	B6_NO	PIN_C10	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[4]	Output	PIN_B21	6	B6_NO	PIN_J12	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[3]	Output	PIN_A21	6	B6_NO	PIN_C11	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[2]	Output	PIN_B19	7	B7_NO	PIN_A12	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[1]	Output	PIN_A20	7	B7_NO	PIN_E10	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min0[0]	Output	PIN_B20	6	B6_NO	PIN_A13	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[6]	Output	PIN_E17	6	B6_NO	PIN_E11	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[5]	Output	PIN_D19	6	B6_NO	PIN_A10	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[4]	Output	PIN_C20	6	B6_NO	PIN_D13	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[3]	Output	PIN_C19	7	B7_NO	PIN_A11	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[2]	Output	PIN_E21	6	B6_NO	PIN_E12	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[1]	Output	PIN_E22	6	B6_NO	PIN_D14	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_min1[0]	Output	PIN_F21	6	B6_NO	PIN_D12	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[6]	Output	PIN_C17	7	B7_NO	PIN_D15	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[5]	Output	PIN_D17	7	B7_NO	PIN_A20	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[4]	Output	PIN_E16	7	B7_NO	PIN_A17	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[3]	Output	PIN_C16	7	B7_NO	PIN_J13	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[2]	Output	PIN_C15	7	B7_NO	PIN_C18	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[1]	Output	PIN_E15	7	B7_NO	PIN_A19	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec0[0]	Output	PIN_C14	7	B7_NO	PIN_B16	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[6]	Output	PIN_B17	7	B7_NO	PIN_H12	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[5]	Output	PIN_A18	7	B7_NO	PIN_A7	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[4]	Output	PIN_A17	7	B7_NO	PIN_A9	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[3]	Output	PIN_B16	7	B7_NO	PIN_B8	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[2]	Output	PIN_E18	6	B6_NO	PIN_B11	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[1]	Output	PIN_D18	6	B6_NO	PIN_B12	2.5 V...ault		12mA...ult)	2 (default)		
seg_dis_sec1[0]	Output	PIN_C18	7	B7_NO	PIN_J11	2.5 V...ault		12mA...ult)	2 (default)		
<<new node>>											

Named: * Edit

Filter: Pins: all

All Pins

0% 00:00:00

图 4.7 数字钟 clockb 引脚分配

4.3 DE10-Lite 开发板综合设计实验 – 步进电机脉冲分配器

4.3.1 步进电机工作原理

步进电机是将电脉冲信号转变为角位移或线位移的开环控制电机，是现代数字程序控制系统中的主要执行元件，应用极为广泛。电机的转速、停止的位置只取决于脉冲信号的频率和脉冲数，当步进驱动器接收到一个脉冲信号，它就驱动步进电机按设定的方向转动一个固定的角度，称为“步距角”，它的旋转是以固定的角度一步一步运行的。可以通过控制脉冲个数来控制角位移量，从而达到准确定位的目的；同时可以通过控制脉冲频率来控制电机转动的速度和加速度，从而达到调速的目的。

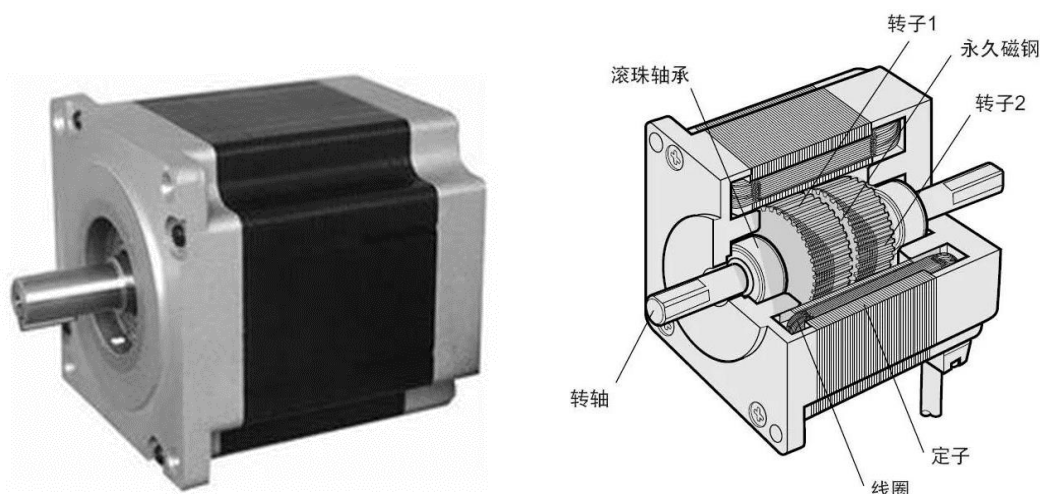


图 4.3.1 步进电机外形及结构剖面图

当 A 相绕组通以直流电流时，根据电磁学原理，便会在 AA 方向上产生一磁场，在磁场电磁力的作用下，吸引转子，使转子的齿与定子 AA 磁极上的齿对齐。若 A 相断电，B 相通电，这时新磁场的电磁力又吸引转子的两极与 BB 磁极齿对齐，转子沿顺时针转过 60° 。通常，步进电机绕组的通断电状态每改变一次，其转子转过的角度 α 称为步距角。如图所示步进电机的步距角 $\alpha = 60^\circ$ 。如果控制线路不停地按 $A \rightarrow B \rightarrow C \rightarrow A \rightarrow \dots$ 的顺序控制步进电机绕组的通断电，步进电机的转子便不停地顺时针转动。若通电顺序改为 $A \rightarrow C \rightarrow B \rightarrow A \rightarrow \dots$ ，同理，步进电机的转子将逆时针不停地转动。

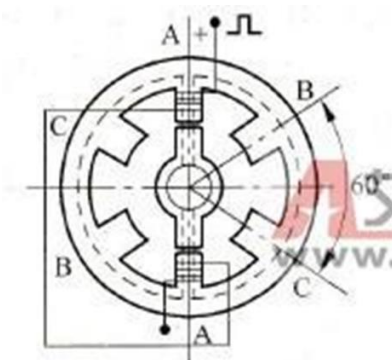


图 4.3.2 步进电机工作原理

上面所述的通电方式称为三相三拍。还有一种三相六拍的通电方式，通电顺序是：顺时针为 $A \rightarrow AB \rightarrow B \rightarrow BC \rightarrow C \rightarrow CA \rightarrow A \rightarrow \dots$ ；逆时针为 $A \rightarrow AC \rightarrow C \rightarrow CB \rightarrow bB \rightarrow BA \rightarrow A \rightarrow \dots$ 。若以三相六拍通电方式工作，当 A 相通电转为 A 和 B 同时通电时，转子的磁极将同时受到 A 相绕组产生的磁场和 B 相绕组产生合成磁场的共同吸引，转子的磁极只好停在 A 和 B 两相磁极之间，这时它的步距角 $\alpha = 30^\circ$ 。当由 A 和 B 两相同时通电转为 B 相通电时，转子

磁极再沿顺时针旋转 30° ，与 B 相磁极对齐。其余依此类推。采用三相六拍通电方式，可使步距角 α 缩小一半。

实际三相反应式步进电机的剖面图如图 4.3.3。电机的定子上有 6 个均布的磁极,组成 A、B、C 三相绕组。转子上均布 40 个小齿,每个齿的齿距为 9° ，每个磁极的极弧上有 5 个小齿,齿距也为 9° 。在制造工艺上保证当 A 相通电,其小齿与转子的的小齿对齐时,B 相和 C 相的小齿分别和转子的的小齿错位 $1/3$ 的齿距,即 $+3^\circ$ 和 -3° 。当 B 相通电时,转子受反应转矩的作用而转动,直到转子上的小齿与 B 磁极上的小齿对齐,转子转过 3° ，此时 A、C 磁极下的小齿又分别与转子的的小齿错开 $1/3$ 齿距。当三相绕组按 $A \rightarrow B \rightarrow C \rightarrow A \rightarrow \dots$ 的顺序循环通电,转子会按顺时针方向以每个通电脉冲转动 3° 的规律步进转动起来。若改变通电顺序,按 $A \rightarrow C \rightarrow B \rightarrow A \rightarrow \dots$ 顺序循环通电,转子将逆时针方向选择。

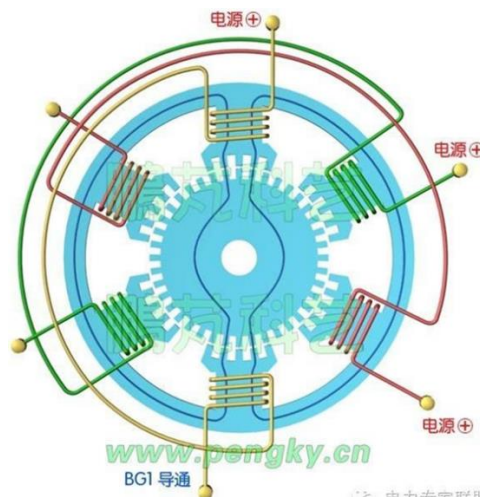


图 4.3.3 实际三相反应式步进电机工作原理

4.3.2 步进电机控制器 FPGA 程序设计要求

- 1、采用 DE10-Lite FPGA 学习板；
- 2、两位数码管（HEX1、HEX0）显示脉冲频率，频率范围：1Hz -- 99Hz；
- 3、频率调节按钮（KEY1，KEY0）；
- 4、工作模式为三相六拍；
- 5、滑动开关 SW0 可控制 运行/停止 状态；
- 6、滑动开关 SW1 可控制 正转/反转 状态；
- 7、A、B、C 三相脉冲及频率 f 采用学习板上 GPIO（JP1）输出
A - GPIO_[1]; B - GPIO_[5]; C - GPIO_[9]; f - GPIO_[11]; GND - PIN12 。

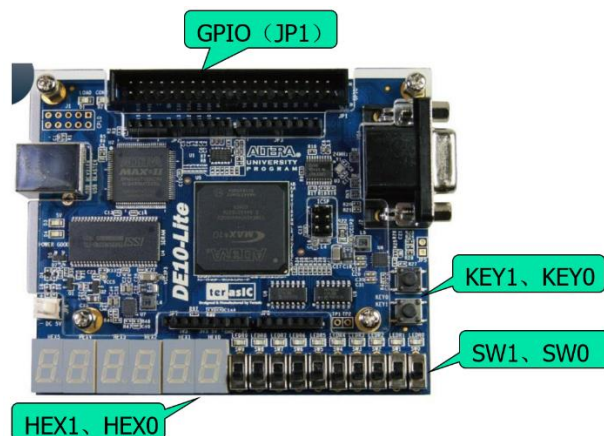


图 4.3.4 FPGA 开发板功能要求

4.3.3 步进电机控制器 FPGA 程序结构说明

步进电机控制器脉冲分配器程序结构如图 4.3.5 所示。

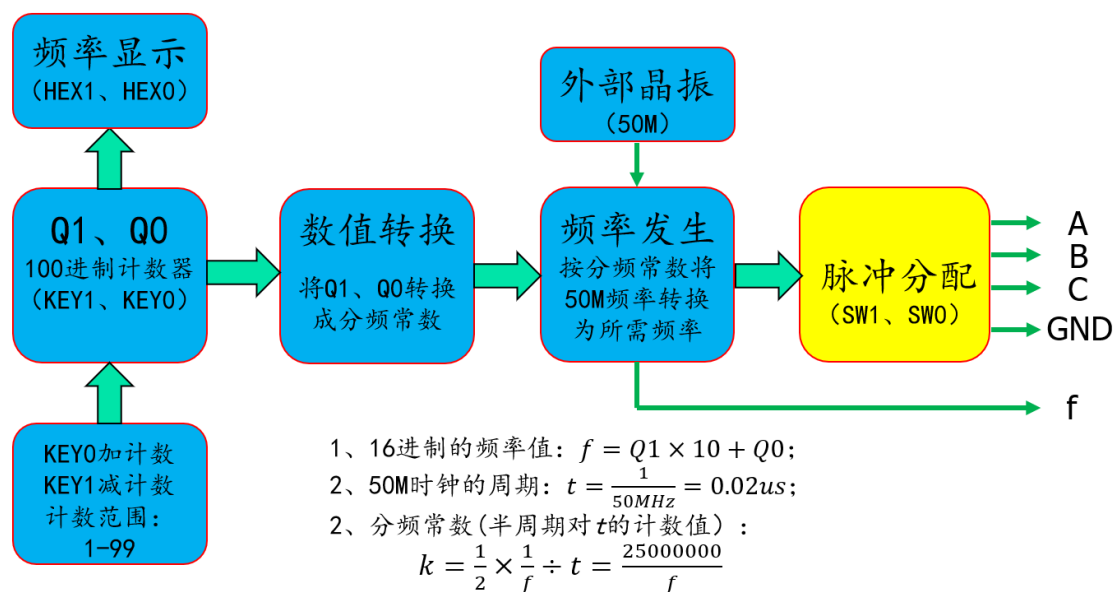


图 4.3.5 步进电机控制器脉冲分配器程序结构图

程序设计参考：

1、100 进制计数器可以参考 clockb 程序设计中，小时和分钟的调整程序部分，计数范围从 1-99。（频率值不能为 0，当为 0 时除法会发生错误。）

2、数值转换程序：

```
q2(3 downto 0) <= q0;    --q2=q0
q3(4 downto 1) <= q1;    --q3=q1x2
q4(6 downto 3) <= q1;    --q4=q1x8
q5 <= q2+q3+q4;         --q5=q1x(2+8)+q0

div_data <= 25000000/conv_integer(q5);  --分频常数
```

3、频率发生程序可以参考秒信号发生程序。

4、脉冲分配程序自己设计。

4.3.4 步进电机控制器 FPGA 程序设计实验报告要求

1、设计目的：

步进电机工作原理，FPGA 程序要求。

2、程序结构：

由多少个程序模块构成，每个模块的功能要求。

3、程序说明：

每个模块的程序及注释。

4、测试结果：

频率设置与输出是否正确，脉冲输出是否正确。

5、问题分析：

针对实验过程中出现的问题进行分析讨论。

6、实验体会。

结合本次实验总结设计的一般规律和步骤，以及调试的方法。说说对对本次实验的体会及意见。